

MIPS Instruction Format

Decision Instructions

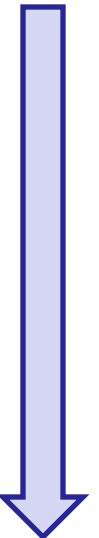
Machine Language -- MIPS

- All instructions have the same length (32 bits)
- **MIPS Instructions:** Three different formats
 - R: arithmetic instruction format
 - I: transfer, branch, immediate format
 - J: jump instruction format
- `add $t0, $s1, $s2`
 - 32 bits in machine language
 - Fields for:
 - Operation (add)
 - Operands (\$s1, \$s2, \$t0)

`A[300] = h + A[300];`

`lw $t0, 1200($t1)`
`add $t0, $s2, $t0`
`sw $t0, 1200($t1)`

101011010010100000000010010110000
00000010010010000100000000100000
100011010010100000000010010110000



Instruction Formats

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R:	op	rs	rt	rd	shamt	funct

I:	op	rs	rt	address / immediate		
-----------	----	----	----	---------------------	--	--

J:	op	target address				
-----------	----	----------------	--	--	--	--

op: basic operation of the instruction (opcode)

rs: first source operand register

rt: second source operand register

rd: destination operand register

shamt: shift amount

funct: selects the specific variant of the opcode (function code)

address: offset for load/store instructions ($\pm 2^{15}$)

immediate: constants for immediate instructions

R Format

add \$t0, \$s1, \$s2 (add \$8, \$17, \$18 # \$8 = \$17 + \$18)

6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

sub \$t1, \$s1, \$s2 (sub \$9, \$17, \$18 # \$9 = \$17 - \$18)

6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
0	17	18	9	0	34
000000	10001	10010	01001	00000	100010

I Format

lw \$t0, 52(\$s3)

lw \$8, 52(\$19)

6 bits

5 bits

5 bits

16 bits

35

19

8

52

100011

10011

01000

0000 0000 0011 0100

sw \$t0, 52(\$s3)

sw \$8, 52(\$19)

6 bits

5 bits

5 bits

16 bits

43

19

8

52

101011

10011

01000

0000 0000 0011 0100

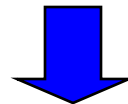
Example

`A[300] = h + A[300];` `/* $t1 <= base of array A; $s2 <= h */`



Compiler

```
lw  $t0, 1200($t1)    # temporary register $t0 gets A[300]
add $t0, $s2, $t0      # temporary register $t0 gets h + A[300]
sw  $t0, 1200($t1)    # stores h + A[300] back into A[300]
```



Assembler

35	9	8	1200
0	18	8	8 0 32
43	9	8	1200



100011	01001	01000	0000 0100 1011 0000
000000	10010	01000	01000 00000 100000
101011	01001	01000	0000 0100 1011 0000

Immediates (Numerical Constants)

- Small constants are used frequently (50% of operands)
 - $A = A + 5;$
 - $C = C - 1;$
- Solutions
 - Put typical constants in memory and load them
 - Create hardwired registers (e.g. **\$0** or **\$zero**)
- MIPS instructions for constants (I format)
 - `addi $t0, $s7, 4` # \$t0 = \$s7 + 4

8	23	8	4
001000	10111	01000	0000 0000 0000 0100

Arithmetic Overflow

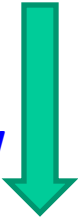
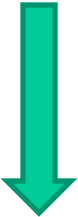
- Computers have *limited arithmetic precision* (32 bits)

$$\begin{array}{r} 15 \\ + 3 \\ \hline 18 \end{array} \qquad \begin{array}{r} 1111 \\ 0011 \\ \hline 10010 \end{array}$$


- Some languages detect **overflow** (Ada), some don't (C)
- MIPS** provides 2 types of arithmetic instructions:
 - Add, sub, and addi: cause overflow
 - Addu, subu, and addiu: (**U**nsigned) do not cause overflow
- MIPS C** compilers produce addu, subu, addiu by default

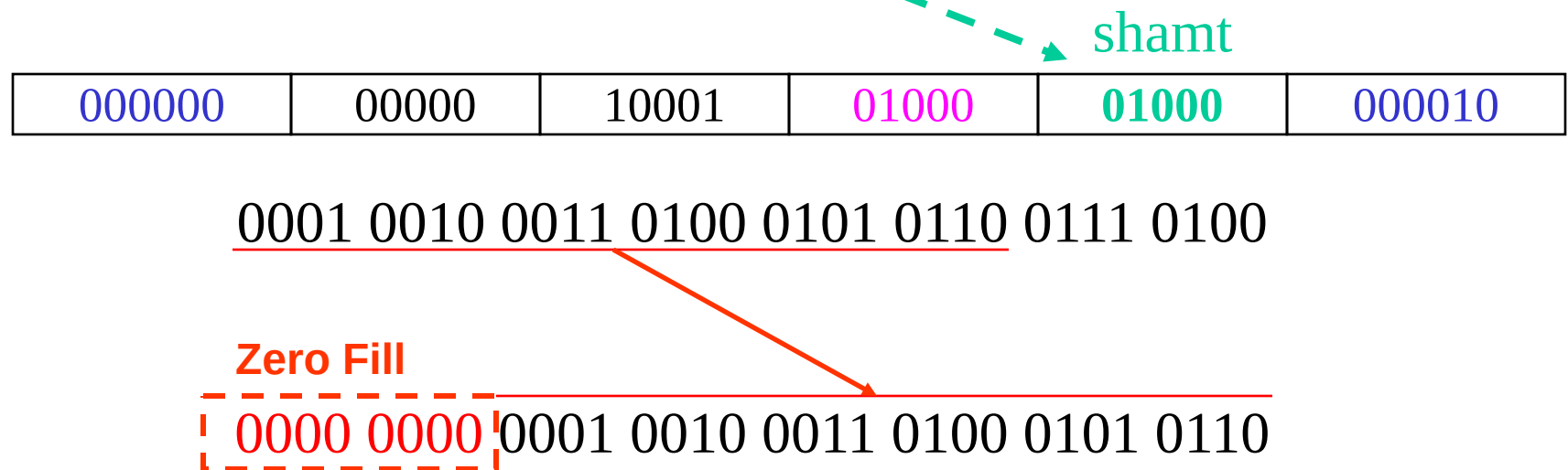
Logical Instructions

- **Bitwise operations**
 - View contents of registers as 32 bits rather than as a single 32-bit number
- **MIPS Instructions**
 - and, or: the 3 operands are registers (R format)
 - andi, ori: the 3rd argument is an immediate (I format)
- **Example: *masking* (`andi $t0, $t0, 0xFFF`)**

B I T W I S E		1011 0110 1010 0100 0011 1101 1001 1010		B I T W I S E
		0000 0000 0000 0000 0000 1111 1111 1111		
		0000 0000 0000 0000 0000 1101 1001 1010		

Shift Instructions

- **Move all the bits** in a register to the left | right
 - **sll** (shift left logical): fills emptied bits with 0s
 - **srl** (shift right logical): fills emptied bits with 0s
- Example: **srl** \$t0, \$s1, **8** (R format)



Multiplication and Division

- Use **special purpose registers** (**hi**, **lo**)

- 32-bit value x 32-bit value = 64-bit value

- **Mult** \$s0, \$s1

000000	10000	10001	00000	00000	011000
--------	-------	-------	-------	-------	--------

- hi: upper half of product

- lo: lower half of product

- **Div** \$s0, \$s1

000000	10000	10001	00000	00000	011010
--------	-------	-------	-------	-------	--------

- hi: remainder (\$s0 / \$s1)

- lo: quotient (\$s0 % \$s1)

- **Move results** into general purpose registers:

- **mfhi** \$s0

000000	00000	00000	10000	00000	010000
--------	-------	-------	-------	-------	--------

- **mflo** \$s1

000000	00000	00000	10001	00000	010010
--------	-------	-------	-------	-------	--------

Assembly vs. Machine Language

- **Assembly provides convenient symbolic representation**
 - Much easier than writing binary numbers
 - **Destination operand first**
 - Pseudo instructions
 - **Labels** to identify and name words that hold instructions/data
- **Machine language is the underlying reality**
 - Destination operand is no longer first
 - Efficient format
- Assembly language can provide **pseudo instructions**
 - **Move** \$t0, \$t1 (add \$t0, \$t1, \$zero)
- When considering performance in terms of instruction count (IC) you should count real (**not pseudo**) instructions

Conditional Branches

- Decision-Making Instructions
- Branch if equal
 - **beq** register1, register2, *destination_address*
- Branch if not equal
 - **bne** register1, register2, *destination_address*
- Example: beq \$s3, \$s4, 20

6 bits	5 bits	5 bits	16 bits
4	19	20	5
000100	10011	10100	0000 0000 0000 0101

Labels

- No need to calculate addresses for branches

if (i == j) go to L1;

f = g + h;

L1: f = f - i;

f => \$s0

g => \$s1

h => \$s2

i => \$s3

j => \$s4

(4000) beq \$s3, \$s4, L1 # if i equals j go to L1

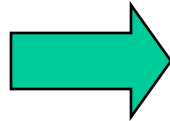
(4004) add \$s0, \$s1, \$s2 # f = g + h

L1: (4008) sub \$s0, \$s0, \$s3 # f = f - i

L1 corresponds to the address of the subtract instruction

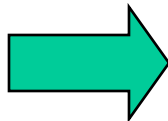
If Statements

```
if (condition)
    clause1;
else
    clause2;
```



```
if (condition) goto L1;
    clause2;
    goto L2;
L1: clause1;
L2:
```

```
if (i == j)
    f = g + h;
else
    f = g - h;
```



```
    beq $3, $4, True
    sub $0, $s1, $s2
    j False
True: add $s0, $s1, $s2
False:
```

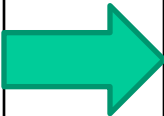
Loops

```
Loop: g = g + A[i];  
      i = i + j;  
      if (i != h) goto Loop;
```

Clever method of
multiplying by 4 to
get byte offset for
one word



```
g: $s1  
h: $s2  
i: $s3  
j: $s4  
Base of A: $s5
```



```
Loop: add $t1, $s3 $s3 # $t1 = 2 * i  
      add $t1, $t1, $t1 # $t1 = 4 * I  
      add $t1, $t1, $s5 # $t1=address of A[i]  
      lw  $t0, 0($t1)  # $t0 = A[i]  
      add $s1, $s1, $t0 # g = g + A[i]  
      add $s3, $s3, $s4 # i = i + j  
      bne $s3, $s2, Loop # go to Loop if i != h
```

Basic Block

Other Inequalities < >

- Programs need to test < and >
- ***Set on less than*** instruction
- **slt** register1, register2, register3
 - register1 = (register2 < register3)? 1 : 0;
- Example: if (g < h) goto Less;

g: \$s0
h: \$s1

slt \$t0, \$s0, \$s1
bne \$t0, \$0, Less

- **slti**: useful in for loops if (g >= 1) goto Loop

slti \$t0, \$s0, 1	# \$t0 = 1 if g < 1
beq \$t0, \$0, Loop	# goto Loop if g >= 1

Relative Conditions

- == != < <= > >=
- MIPS does not support directly the last four
- Compilers use **slt**, **beq**, **bne**, **\$zero**, and **\$at**
- Pseudoinstructions
 - **blt \$t1, \$t2, L** # if (\$t1 < \$t2) go to L {
 slt \$at, \$t1, \$t2
 bne \$at, \$zero, L
 - **ble \$t1, \$t2, L** # if (\$t1 <= \$t2) go to L {
 slt \$at, \$t2, \$t1
 beq \$at, \$zero, L
 - **bgt \$t1, \$t2, L** # if (\$t1 > \$t2) go to L {
 slt \$at, \$t2, \$t1
 bne \$at, \$zero, L
 - **bge \$t1, \$t2, L** # if (\$t1 >= \$t2) go to L {
 slt \$at, \$t1, \$t2
 beq \$at, \$zero, L

Switch Statement

```
switch (k) {  
  case 0: f = i + j; break;  
  case 1: f = g + h; break;  
  case 2: f = g - h; break;  
  case 3: f = i - j; break;  
}
```

```
if (k==0) f = i + j;  
else if (k==1) f = g + h;  
else if (k==2) f = g - h;  
else if (k==3) f = i - j;
```

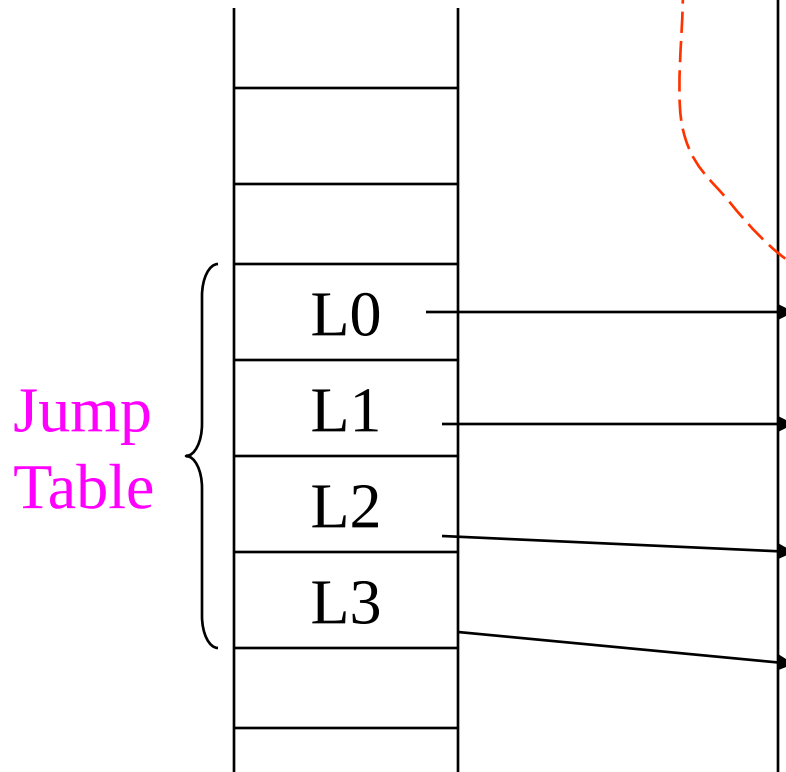
```
# f: $s0; g: $s1; h: $s2; i: $s3; j: $s4; k:$s5
```

```
    bne  $s5, $0, L1      # branch k != 0  
    add  $s0, $s3, $s4    # f = i + j  
    j     Exit            # end of case  
L1: addi $t0, $s5, -1     # $t0 = k - 1  
    bne  $t0, $0, L2      # branch k != 1  
    add  $s0, $s1, $s2    # f = g + h  
    j     Exit            # end of case  
L2: addi $t0, $s5, -2     # $t0 = k - 2  
    bne  $t0, $0, L3      # branch k != 2  
    sub  $s0, $s1, $s2    # f = g - h  
    j     Exit            # end of case  
L3: addi $t0, $s5, -3     # $t0 = k - 3  
    bne  $t0, $0, Exit    # branch k != 3  
    sub  $s0, $s3, $s4    # f = i - j  
Exit:
```

Jump Tables

- Jump register instruction

- **jr** <register>
- unconditional branch to address contained in register



```
# f: $s0; g: $s1; h: $s2; i: $s3; j: $s4; k:$s5
```

```
# $t2 = 4; $t4 = base address of JT
```

```
slt    $t3, $s5, $zero    # test k < 0
bne    $t3, $zero, Exit   # if so, exit
slt    $t3, $s5, $t2      # test k < 4
beq    $t3, $zero, Exit   # if so, exit
add    $t1, $s5, $5       # $t1 = 2*k
add    $t1, $t1, $t1      # $t1 = 4*k
add    $t1, $t1, $t4      # $t1 = &JT[k]
lw     $t0, 0($t1)        # $t0 = JT[k]
jr     $t0              # jump register
```

```
L0: add $s0, $s3, $s4      # k == 0
      j  Exit             # break
L1: add $s0, $1, $s2       # k == 1
      j  Exit             # break
L2: sub $s0, $s1, $s2      # k == 2
      j  Exit             # break
L3: sub $s0, $s3, $s4      # k == 3
Exit:
```

Conclusions

- **MIPS instruction format** – 32 bits
- **Assembly:** Destination = first operand
- **Machine Language:** Dest = last operand
- **Three MIPS formats:**
 - R (arithmetic)**
 - I (immediate)**
 - J (jump)**
- **Decision instructions** – use *jump* (goto)
- *Improve Performance:* loop unrolling